

Estrutura de Dados 2

Prof. Igor Calebe Zadi
igor.zadi@ifsp.edu.br



INSTITUTO FEDERAL
São Paulo
Campus Catanduva



I. Complexidade de Algoritmos



I. Fundamentos de Estruturas de Dados

1. Introdução à análise de algoritmos
2. Crescimento de funções
3. Notações assintóticas
4. Análise de algoritmos iterativos e recursivos
5. Pior, melhor e caso médio
6. Classes de complexidade: P, NP
7. Tabelas de crescimento e desempenho

4. Análise de algoritmos iterativos e recursivos

4. Análise de algoritmos iterativos e recursivos

- Revisão:
 - O que é um algoritmo?
 - Uma sequência de passos para resolver um problema.
 - O que é complexidade de algoritmos?
 - Uma medida da eficiência de um algoritmo.
 - Notação Big-O (O)?
 - Uma forma de descrever a taxa de crescimento do tempo de execução de um algoritmo à medida que o tamanho da entrada (n) aumenta.
 - Exemplos: $O(1)$, $O(\log n)$, $O(n)$, $O(n^2)$, $O(2^n)$.
 - Por que isso importa?
 - Para escolher o melhor algoritmo para um problema.



4. Análise de algoritmos iterativos e recursivos

- Iterativos

- Algoritmos que usam laços de repetição (for, while).
- A execução é sequencial e controlada por uma variável de loop.
 - Exemplo: somar todos os elementos de um vetor.
- Como analisar a complexidade?
 - Conte o número de operações básicas dentro do loop.
 - Determine quantas vezes o loop é executado.
 - O tempo total é a multiplicação desses dois valores.

4. Análise de algoritmos iterativos e recursivos

- Exemplo 1: Qual a complexidade de tempo do algoritmo que imprime os números de 1 até n :

```
n := 5;  
para i até n faça  
    escreva(i);
```

O loop roda n vezes $\rightarrow$ complexidade $O(n)$

4. Análise de algoritmos iterativos e recursivos

- Exemplo 2: Qual a complexidade de tempo do algoritmo?

```
n := 4;  
para i até n:  
    para j até n:  
        S;
```

O loop externo roda n vezes.

Para cada i , o loop interno roda n vezes.

Total: $n * n = n^2 \rightarrow$ complexidade $O(n^2)$.

4. Análise de algoritmos iterativos e recursivos

- Exercício: Qual a complexidade do algoritmo?

```
função fatorial(n):  
    resultado := 1;  
    para i até n faça  
        resultado := resultado * i;  
    retorne resultado;
```



4. Análise de algoritmos iterativos e recursivos

- Algoritmos recursivos:
 - Uma função que chama a si mesma.
 - Possuem duas partes essenciais:
 - Caso Base: A condição de parada.
 - Passo Recursivo: A chamada para si mesma com uma entrada menor.

4. Análise de algoritmos iterativos e recursivos

- Relação de recorrência:
 - Usadas para descrever o tempo de execução de algoritmos recursivos.
 - Definição: fórmula que descreve $T(n)$ em termos de $T(n - 1)$, $T(n - 2)$, etc.

4. Análise de algoritmos iterativos e recursivos

- Exemplo 3: Qual a complexidade de tempo do algoritmo?

```
função fatorial_recursivo(n){  
    se n == 0 então  
        retorne 1;  
    se não:  
        retorne n * fatorial_recursivo(n - 1);  
}
```

a equação de recorrência é: $T(n) = T(n - 1) + O(1)$

A complexidade é $O(n)$

4. Análise de algoritmos iterativos e recursivos

- Exemplo 3: Explorando...

Relação de recorrência:

$$\begin{aligned}T(1) \\&= T(0) + c \dots T(n-1) \\&= T(n-2) + cT(n) \\&= T(n-1) + c\end{aligned}$$

Somando tudo:

$$\begin{aligned}T(n) &= T(0) + nc \\&= c(n+1) \text{ Quando } k \\&= n: T(n) \\&= T(n-k) + kcT(n) \\&= T(n-2) + 2cT(n) \\&= T(n-1) + c\end{aligned}$$

5. Pior, melhor e caso médio



5. Pior, melhor e caso médio

- Um algoritmo pode ter um comportamento diferente para entradas de tamanho igual a n .
 - Pior caso (ou pessimista);
 - Melhor caso (ou otimista);
 - Caso médio.



5. Pior, melhor e caso médio

- Um algoritmo pode ter um comportamento diferente para entradas de tamanho igual a n .
 - Pior caso (ou pessimista);
 - Melhor caso (ou otimista);
 - Caso médio.

5. Pior, melhor e caso médio

- Calcule as complexidades de pior caso, melhor caso e caso médio
 - Pior caso (ou pessimista);
 - Melhor caso (ou otimista);
 - Caso médio.

Algoritmo Busca (A, n, x)

Entrada: procura o valor igual a x no vetor “A”, de “ n ” elementos.

Saída: retorna o índice do elemento do vetor “A” igual a “ x ” caso tal elemento não exista.

```
{  
     $i := 1$ ;  
    enquanto (  $i \leq n$  e  $x \neq A[i]$  )  
         $i := i + 1$ ;  
  
    se (  $i \leq n$  )  
        retornar  $i$   
    senão  
        retornar 0  
}
```



5. Pior, melhor e caso médio

- Melhor caso:
 - Ocorre quando $x = A[1]$
 - $T(n) = 1$



5. Pior, melhor e caso médio

- Pior caso:
 - Ocorre quando $x = A[n]$ ou se x não está no vetor
 - $T(n) = n$



5. Pior, melhor e caso médio

- Caso médio:
 - É a média entre todos os possíveis casos, supondo uma distribuição estatística da entrada
 - Pensaremos em uma distribuição uniforme – Todos os casos são igualmente prováveis
 - No vetor do exemplo (vetor A) a probabilidade de achar x em qualquer posição é a mesma.

5. Pior, melhor e caso médio

- Caso médio:
 - Para um vetor de tamanho n , a média das posições é :

$$\frac{1+2+3+\dots+n}{n} = \frac{n(n+1)}{2n} = \frac{n+1}{2}$$

- Assim, o tempo esperado para encontrar o valor é proporcional a $(n + 1)/2$, o que também é $O(n)$.

5. Pior, melhor e caso médio

- Caso médio:

Caso	Condição	Complexidade do Caso	Probabilidade
1	Elemento x encontrado na 1ª posição (índice $i=1$)	1	$1/(n+1)$
2	Elemento x encontrado na 2ª posição (índice $i=2$)	2	$1/(n+1)$
3	Elemento x encontrado na 3ª posição (índice $i=3$)	3	$1/(n+1)$
...	...	...	...
n	Elemento x encontrado na n -ésima posição (índice $i=n$)	n	$1/(n+1)$
$n+1$	Elemento x não pertence ao vetor A	n	$1/(n+1)$

5. Pior, melhor e caso médio

- Substituindo da tabela

$$T_{médio}(n) = \left(1 \times \frac{1}{n+1}\right) + \left(2 \times \frac{1}{n+1}\right) + \left(3 \times \frac{1}{n+1}\right) + \dots + \left(n \times \frac{1}{n+1}\right) + \left(n \times \frac{1}{n+1}\right)$$

- Simplificando

$$T_{médio}(n) = \frac{1}{n+1} (1 + 2 + 3 + \dots + n) + \frac{n}{n+1}$$

5. Atividade (em duplas)

Calcule a complexidade dos algoritmos:

Algoritmo Fib (n)

Entrada: n , inteiro, $n \geq 1$.

Saída: retorna o elemento de ordem n da sequência de Fibonacci.

```
{  
  a := 1; aa := 1; f := 1; i := 3;  
  enquanto ( i ≤ n )  
  {  
    aux := f; f := a + aa; aa := a; a := aux;  
    i := i + 1  
  }  
  
  retornar f  
}
```

Algoritmo Fib (n)

Entrada: n , inteiro, $n \geq 1$.

Saída: retorna o elemento de ordem n da sequência de Fibonacci.

```
{  
  se (  $n \leq 2$  )  
    retornar 1  
  senão  
    retornar Fib (  $n - 1$  ) + Fib (  $n - 2$  )  
}
```




Observações sobre o material eletrônico

- O material ficará disponível na pasta compartilhada que é acessada sob convite
- O material foi elaborado a partir de diversas fontes (livros, internet, colegas, alunos etc.)
- Alguns trechos podem ter sido inteiramente transcritos a partir dessas fontes
- Outros trechos são de autoria própria
- Esta observação deve estar presente em qualquer utilização do material fora do ambiente de aulas do IFSP - Catanduva